

SaaS de Gestão de Calibração

Modelagem mínima do banco + stack inicial do backend em Spring Boot

Objetivo: colocar o MVP em produção com o menor número de tabelas e dependências possível, preservando apenas o que é necessário para multiempresa, autenticação, cadastro de instrumentos e histórico de calibrações.

1. Modelo reduzido

O MVP usa **5 tabelas principais**. Foram removidas as tabelas unidade, tipo_instrumento e certificado nesta primeira versão.

```
EMPRESA
|-- USUARIO
|-- SETOR
    |-- INSTRUMENTO
    |-- CALIBRACAO
```

Tabela	Finalidade	Relacionamento principal
empresa	Cliente/tenant do SaaS	Raiz do domínio
usuario	Acesso ao sistema	empresa 1:N usuario
setor	Local lógico do instrumento	empresa 1:N setor
instrumento	Equipamento controlado	setor 1:N instrumento
calibracao	Histórico de calibrações	instrumento 1:N calibracao

Por que manter calibração separada? Porque ela é histórica. Um mesmo instrumento terá várias calibrações ao longo do tempo.

2. Tabelas do MVP

2.1 empresa

```
empresa
-----
id UUID PK
razao_social VARCHAR(150) NOT NULL
nome_fantasia VARCHAR(150)
cnpj VARCHAR(14) UNIQUE
email VARCHAR(150)
telefone VARCHAR(20)
plano VARCHAR(30)
status VARCHAR(20) NOT NULL
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
```

2.2 usuario

```
usuario
-----
id UUID PK
empresa_id UUID FK -> empresa.id
nome VARCHAR(150) NOT NULL
email VARCHAR(150) NOT NULL
senha_hash VARCHAR(255) NOT NULL
perfil VARCHAR(30) NOT NULL
ativo BOOLEAN NOT NULL
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
UNIQUE (empresa_id, email)
```

2.3 setor

```
setor
-----
id UUID PK
empresa_id UUID FK -> empresa.id
nome VARCHAR(100) NOT NULL
descricao VARCHAR(255)
ativo BOOLEAN NOT NULL
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
```

2.4 instrumento

```
instrumento
-----
id UUID PK
empresa_id UUID FK -> empresa.id
setor_id UUID FK -> setor.id
codigo_interno VARCHAR(50) NOT NULL
nome VARCHAR(150) NOT NULL
tipo VARCHAR(100)
fabricante VARCHAR(100)
modelo VARCHAR(100)
numero_serie VARCHAR(100)
faixa_medicao VARCHAR(100)
resolucao VARCHAR(100)
periodicidade_meses INTEGER
criticidade VARCHAR(20)
status_operacional VARCHAR(30) NOT NULL
responsavel VARCHAR(150)
observacao TEXT
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
UNIQUE (empresa_id, codigo_interno)
```

2.5 calibracao

```
calibracao
-----
id UUID PK
empresa_id UUID FK -> empresa.id
instrumento_id UUID FK -> instrumento.id
data_calibracao DATE NOT NULL
data_validade DATE NOT NULL
laboratorio VARCHAR(150)
numero_certificado VARCHAR(100)
resultado VARCHAR(30) NOT NULL
observacao TEXT
custo NUMERIC(12,2)
certificado_storage_key VARCHAR(500)
certificado_nome VARCHAR(255)
created_by UUID FK -> usuario.id
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
```

Decisão do MVP: o certificado não ganha tabela própria; a calibração guarda a referência do arquivo no object storage.

3. Enums mínimos

Enum	Valores iniciais
EmpresaStatus	TESTE, ATIVA, INATIVA, BLOQUEADA
PerfilUsuario	ADMIN, QUALIDADE, CONSULTA
Criticidade	BAIXA, MEDIA, ALTA, CRITICA
StatusOperacional	ATIVO, MANUTENCAO, FORA_DE_USO, DESCARTADO
ResultadoCalibracao	APROVADO, REPROVADO, APROVADO_COM_RESTRICAO

4. Regras importantes

- Multiempresa: toda consulta deve respeitar empresa_id do tenant autenticado.
- Certificado: guardar arquivo em object storage e somente a referência no PostgreSQL.
- Status metrológico: calcular pela calibração mais recente e data_validade.
- Status operacional do instrumento permanece separado do status da calibração.
- Usar Flyway desde o início e evitar ddl-auto=update em produção.
- Usar UUID nas chaves principais.

5. Índices iniciais

```
CREATE INDEX idx_usuario_empresa ON usuario (empresa_id);
CREATE INDEX idx_setor_empresa ON setor (empresa_id);
CREATE INDEX idx_instrumento_empresa_setor ON instrumento (empresa_id, setor_id);
CREATE INDEX idx_calibracao_instrumento_data ON calibracao (instrumento_id, data_calibracao DESC);
CREATE INDEX idx_calibracao_empresa_validade ON calibracao (empresa_id, data_validade);
```

6. Stack mínima

Java 21 + Spring Boot + PostgreSQL + Flyway + Docker

Dependência	Uso
Spring Web	REST API
Spring Data JPA	ORM/repositories
PostgreSQL Driver	Conexão com o banco
Validation	Validação de DTOs
Spring Security	Autenticação/autorização
Flyway	Migrations
Spring Boot Test	Testes

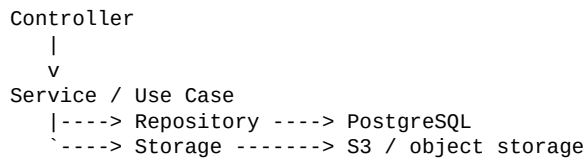
Fora do MVP: Redis, Kafka/RabbitMQ, MongoDB, Elasticsearch, WebFlux, microserviços, Kubernetes, Spring Cloud e event sourcing.

7. Estrutura de pacotes

```
com.astralis.calibracao
|-- auth
|-- empresa
|-- usuario
|-- setor
|-- instrumento
|-- calibracao
|-- storage
|-- security
|-- exception
`-- config
```

8. Arquitetura

Monólito modular: um único backend Spring Boot e um único PostgreSQL.



9. Ordem prática para implementar

1. Criar projeto Spring Boot com Java 21 e Maven.
2. Subir PostgreSQL via Docker Compose.
3. Configurar Flyway.
4. Criar migration V1 com as 5 tabelas.
5. Implementar empresa e autenticação.
6. Implementar CRUD de setor.
7. Implementar CRUD de instrumento.
8. Implementar histórico de calibrações.
9. Adicionar upload do certificado.
10. Criar dashboard simples.
11. Adicionar testes antes de expandir.

10. O que entra depois, somente se houver necessidade

Necessidade	Evolução
Várias filiais/plantas	Criar unidade
Tipos personalizados por cliente	Criar tipo_instrumento
Vários documentos por calibração	Criar certificado/anexo

11. Resumo

MVP final: 5 tabelas.

empresa -> usuario
empresa -> setor -> instrumento -> calibracao

Suficiente para autenticar clientes, separar empresas, organizar instrumentos por setor, manter histórico de calibração e anexar certificado.

Versão reduzida da modelagem original para foco exclusivo no MVP.